Web Programming Report

Gebeya Multi-Role E-Commerce Marketplace

Full-stack PHP/MySQL marketplace with buyer, seller, and admin roles, Multi-Factor Authentication, session management, and a full order lifecycle.

Overview

Gebeya is a full-stack e-commerce marketplace built with PHP and MySQL on XAMPP. The platform supports three distinct user roles; buyer, seller, and admin each with their own dashboard and access controls. Buyers browse and purchase products; sellers list and fulfil orders; admins moderate the catalogue, manage users, and maintain categories. A Multi-Factor Authentication system protects every account, requiring both a password and an emailed one-time code before any protected area can be accessed.

The application follows a layered architecture: a shared includes/ layer handles configuration, database access, authentication, CSRF protection, and the flash-message system. Public-facing pages live in public/, with separate folder hierarchies for seller/ and admin/ functionality. PIain HTML, CSS, and vanilla JavaScript power the frontend with no framework. All database interaction uses MySQLi prepared statements, and user-supplied data is passed through htmlspecialchars before rendering.

Technology Stack

Gebeya is built on a conventional LAMP-style stack running under XAMPP. The backend language is PHP 8, using the MySQLi extension with exception mode enabled via MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT for all database access. Composer manages a single third-party dependency, PHPMailer, used for SMTP delivery of one-time passwords.

Component	Purpose
PHP 8 / XAMPP	Server-side scripting and runtime environment
MySQL (via MySQLi)	Relational database; all queries use prepared statements
PHPMailer 7.x	SMTP email delivery for OTP codes via Mailtrap sandbox
Composer	Dependency manager; entry point via vendor/autoload.php
Native PHP sessions	Server-side session storage; cookie settings hardened in config.php
htmlspecialchars / strip_tags	Output encoding via the global e() helper; input tag stripping on names
finfo (MIME detection)	Binary image type verification in the file upload pipeline
password_hash / password_verify	PHP's built-in bcrypt wrappers for credential and OTP hashing
random_int / random_bytes	Cryptographically secure OTP and CSRF token generation

System Architecture

Gebeya follows a three-tier architecture. The browser communicates with a PHP server over HTTP, which reads from and writes to a MySQL database. All page loads are full server-rendered responses; there is no AJAX layer; form submissions use standard HTTP POST and redirect on success.

Request Pipeline

Every PHP page bootstraps the same shared layer before any business logic runs. The standard include order is:

- includes/config.php sets session cookie flags (HttpOnly, SameSite=Strict) and emits global security headers.
- includes/db.php reads .env credentials, enables MYSQLI_REPORT_STRICT, and establishes the MySQLi connection with utf8mb4 charset.
- includes/auth.php starts the session, loads CSRF helpers, and exposes require_login(), require_role(), and require_buyer_only().
- includes/functions.php provides e(), redirect(), flash_set()/flash_get_all(), has_role(), and cart cookie sync logic.

```php
<?php
// includes/db.php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);

$env_file = __DIR__ . '/../.env';
if (file_exists($env_file)) {
    $lines = file($env_file, FILE_IGNORE_NEW_LINES | FILE_SKIP_EMPTY_LINES);
    foreach ($lines as $line) {
        if (strpos(trim($line), '#') === 0) continue;
        $parts = explode('=', $line, 2);
        if (count($parts) === 2) {
            $_ENV[trim($parts[0])] = trim($parts[1]);
        }
    }
}

$DB_HOST = $_ENV['DB_HOST'] ?? "127.0.0.1";
$DB_USER = $_ENV['DB_USER'] ?? "root";
$DB_PASS = $_ENV['DB_PASS'] ?? "";
$DB_NAME = $_ENV['DB_NAME'] ?? "gebeya_db";

try {
    $conn = mysqli_connect($DB_HOST, $DB_USER, $DB_PASS, $DB_NAME);
    mysqli_set_charset($conn, "utf8mb4");
} catch (mysqli_sql_exception $e) {
    die("Database connection failed: " . $e->getMessage());
}
```

Figure 1: *Database bootstrap enabling strict MySQLi error reporting, parsing credentials from the .env file, and establishing the connection with utf8mb4 charset*

Directory Layout

File / Folder	Purpose
includes/config.php	Session security settings and global HTTP security headers
includes/db.php	Environment variable loader and MySQLi connection factory
includes/auth.php	Login enforcement, role guards, CSRF generation and verification
includes/functions.php	Global helpers: output escaping, redirect, flash messages, role checks, cart sync
includes/mailer.php	PHPMailer wrapper sends OTP emails via Mailtrap SMTP sandbox
includes/header.php / footer.php	Shared navigation and layout wrappers included on every page
public/	Buyer-facing pages: homepage, store, product, login, register, OTP verify, cart, checkout, orders, profile
seller/	Seller-only pages: dashboard, create/edit/delete listings, orders
admin/	Admin-only pages: dashboard, user list, listings moderation, orders overview, category management
sql/schema.sql	Full database schema with tables, indexes, foreign keys, and constraints
sql/seed.sql	Sample data for development and testing
.env	Runtime secrets: DB credentials and Mailtrap SMTP settings (blocked from web by .htaccess)
vendor/	Composer-managed dependencies (PHPMailer)

Database Design

The MySQL database (gebeya_db, utf8mb4_unicode_ci) contains tables that collectively support authentication, the product catalogue, shopping cart logic, order fulfilment, and reviews. Foreign keys with ON DELETE CASCADE and ON DELETE RESTRICT are used throughout to maintain referential integrity.

```sql
CREATE TABLE users (
  user_id INT AUTO_INCREMENT PRIMARY KEY,
  full_name VARCHAR(120) NOT NULL,
  email VARCHAR(190) NOT NULL UNIQUE,
  password_hash VARCHAR(255) NOT NULL,
  phone VARCHAR(40) NULL,
  status ENUM('active','pending','banned') NOT NULL DEFAULT 'active',
  otp_hash VARCHAR(255) NULL,
  otp_expires_at DATETIME NULL,
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;

CREATE TABLE user_roles (
  user_id INT NOT NULL,
  role_id INT NOT NULL,
  PRIMARY KEY (user_id, role_id),
  CONSTRAINT fk_user_roles_user
    FOREIGN KEY (user_id) REFERENCES users(user_id)
    ON DELETE CASCADE ON UPDATE CASCADE,
  CONSTRAINT fk_user_roles_role
    FOREIGN KEY (role_id) REFERENCES roles(role_id)
    ON DELETE RESTRICT ON UPDATE CASCADE
) ENGINE=InnoDB;

CREATE TABLE products (
  product_id INT AUTO_INCREMENT PRIMARY KEY,
  seller_id INT NOT NULL,
  title VARCHAR(160) NOT NULL,
  description TEXT NULL,
  price DECIMAL(10,2) NOT NULL,
  stock INT NOT NULL DEFAULT 0,
  product_condition ENUM('new','used') NOT NULL DEFAULT 'used',
  status ENUM('active','pending','hidden') NOT NULL DEFAULT 'active',
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_products_seller
    FOREIGN KEY (seller_id) REFERENCES users(user_id)
    ON DELETE RESTRICT ON UPDATE CASCADE,
  INDEX idx_products_title (title),
  INDEX idx_products_seller (seller_id),
```

Figure 2: *Schema definitions for the users, user_roles, and products tables, showing OTP columns, composite primary key for role assignment, and product status enum.*

Tables and Their Roles

Table	Key Columns	Purpose
roles	role_id, role_name (UNIQUE)	Defines the three platform roles: buyer, seller, admin

users	user_id, email (UNIQUE), password_hash, otp_hash, otp_expires_at, status	Core account table. Stores bcrypt password hash and temporary OTP hash with 5-minute expiry
user_roles	user_id + role_id (composite PK)	Many-to-one join enforcing single-role assignment; cascades on user delete
categories	category_id, category_name (UNIQUE)	Admin-managed product taxonomy
products	product_id, seller_id, title, price, stock, product_condition, status, image_path	Product listings with indexed title, seller_id, and status columns
product_categories	product_id + category_id (composite PK)	Many-to-many join between products and categories; cascades on product delete
addresses	address_id, user_id, label, city, street, house_no, postal_code	User delivery addresses; indexed on user_id; cascades on user delete
orders	order_id, buyer_id, address_id, order_status (ENUM), total_amount	Order header; indexed on buyer_id and order_status
order_items	order_item_id, order_id, product_id, seller_id, quantity, unit_price, line_total, item_status	Line items linking orders to products and sellers; indexed on order_id and seller_id
payments	payment_id, order_id (UNIQUE), payment_method, payment_status, paid_at	One payment record per order; four supported payment methods
reviews	review_id, product_id, buyer_id, rating (CHECK 1-5), comment	One review per buyer per product enforced by a UNIQUE constraint

Key Design Decisions

- Single-role enforcement: the user_roles table physically prevents multiple roles per user by deleting any existing row before inserting during registration.
- OTP stored on the users row: otp_hash and otp_expires_at are columns on users itself, avoiding a join during login. They are set to NULL immediately after successful verification.
- Stock guarding at checkout: the checkout handler uses UPDATE products SET stock = stock - ? WHERE product_id = ? AND stock >= ? inside a transaction. If affected_rows is not 1 the transaction rolls back, preventing overselling.
- Indexed columns: email (UNIQUE on users), products.title, products.status, orders.buyer_id, order_items.seller_id all carry explicit indexes for fast query paths.

Page Reference

Public Pages

Page	Purpose	Access Guard
public/index.php	Homepage landing page and entry point for visitors	None
public/store.php	Browse and search the product catalogue with category filters	None
public/product.php	Individual product detail page with add-to-cart form	None
public/login.php	Step 1 of MFA; email and password form with CSRF token and rate limiting	Must not be logged in
public/verify_otp.php	Step 2 of MFA; 6-digit OTP entry with 3-attempt limit and resend throttle	pending_mfa_user_id in session
public/register.php	Account creation; buyer or seller role, password strength validation, CSRF check	Must not be logged in
public/logout.php	Destroys session, expires session cookie, starts a new session with regenerated ID	None
public/cart.php	Shopping cart; view, update quantities, remove items	require_buyer_only()
public/checkout.php	Order placement; address and payment selection, stock guard, DB transaction	require_buyer_only(), cart non-empty
public/my_orders.php	Order history for the logged-in buyer	require_login()
public/order_detail.php	Single order detail with items and payment status	require_login() + ownership check
public/profile.php	View/edit name and phone; manage delivery addresses	require_login()

Seller Pages

Page	Purpose	Access Guard
seller/seller_dashboard.php	Dashboard showing total listings, active count, and recent listings table	require_role('seller')
seller/listing_form.php	Create a new product listing with image upload, categories, price, and stock	require_role('seller')
seller/listings.php	Full listing table with delete action and image file cleanup	require_role('seller')
seller/edit_listing.php	Edit all fields of an existing listing; ownership enforced by WHERE seller_id=?	require_role('seller')

seller/orders.php	View and update fulfilment status of the seller's own order items	require_role('seller')

Admin Pages

Page	Purpose	Access Guard
admin/admin_dashboard.php	Admin hub with navigation cards to all management sections	require_role('admin')
admin/users.php	Full user list with roles and account status indicators	require_role('admin')
admin/listings.php	Product moderation set each listing to active, hidden, or pending	require_role('admin')
admin/orders.php	All-orders overview joining orders, users, and payments	require_role('admin')
admin/categories.php	Add and delete marketplace categories; deletion blocked by FK RESTRICT if products are linked	require_role('admin')

Authentication and Session Flow

Authentication in Gebeya follows a two-step MFA process. A correct password alone is not sufficient to access any protected page; the user must also enter a one-time code delivered to their registered email address before the session is fully established.

Step-by-Step Login Lifecycle

1. The customer submits the login form with their email, password, and CSRF token. check_csrf() runs first, then the email is validated with filter_var, and a session-based rate limiter checks whether the account is temporarily locked (5 failed attempts within 5 minutes).

2. The password is verified against the stored bcrypt hash using password_verify(). On success, a 6-digit code is generated with random_int(100000, 999999), immediately hashed with password_hash(), and written to users.otp_hash with an otp_expires_at timestamp five minutes in the future.

3. The plain-text OTP is dispatched by PHPMailer to the Mailtrap SMTP sandbox. The user's ID is stored in $_SESSION['pending_mfa_user_id']; a staging key that does not grant access and session_regenerate_id(true) rotates the session ID.

4. On verify_otp.php, the user submits the 6-digit code. A per-session counter ($_SESSION['otp_attempts']) caps verification at 3 attempts per 5 minutes. The submitted code is compared against the stored hash; if expired or mismatched, the attempt is rejected and the counter is incremented.

5. On success, otp_hash and otp_expires_at are set to NULL in the database, the staging key is replaced with $_SESSION['user_id'], and session_regenerate_id(true) fires again. The user is routed to their dashboard based on role.

6. On logout, $_SESSION is set to an empty array, the session cookie is expired by reissuing it with a negative max-age, session_destroy() is called, and a fresh session with a new ID is started.

OTP Generation

```php
// MFA OTP GENERATION
$otp = (string)random_int(100000, 999999);
$otp_hash = password_hash($otp, PASSWORD_DEFAULT);
$expires = date('Y-m-d H:i:s', time() + 300); // 5 minutes expiry

$upd = mysqli_prepare($conn, "UPDATE users SET otp_hash = ?, otp_expires_at = ? WHERE user_id = ?");
mysqli_stmt_bind_param($upd, "ssi", $otp_hash, $expires, $user_id);
mysqli_stmt_execute($upd);
mysqli_stmt_close($upd);

// Send Email
send_otp_email($email, $otp);

// Set pending MFA session
$_SESSION["pending_mfa_user_id"] = (int)$user_id;
session_regenerate_id(true);

// Reset login attempts on successful password verification
$_SESSION['login_attempts'] = 0;
unset($_SESSION['last_login_attempt']);

redirect('/public/verify_otp.php');
```

Figure 3: *OTP generation after successful password verification; a 6-digit code is produced via random_int, immediately bcrypt-hashed, stored with a 5-minute expiry, and the user is placed in a pending MFA session before redirect.*

OTP Verification

```php
} elseif (!password_verify($otp_input, $otp_hash)) {
    $errors[] = "Invalid code.";
    $_SESSION['otp_attempts']++;
} else {
    // SUCCESS!
    // Clear OTP
    $clear = mysqli_prepare($conn, "UPDATE users SET otp_hash = NULL, otp_expires_at = NULL WHERE user_id = ?");
    mysqli_stmt_bind_param($clear, "i", $user_id);
    mysqli_stmt_execute($clear);
    mysqli_stmt_close($clear);

    // Log user in
    $_SESSION['user_id'] = $user_id;
    unset($_SESSION['pending_mfa_user_id']);
    unset($_SESSION['otp_attempts']);
    unset($_SESSION['first_otp_attempt']);
    session_regenerate_id(true);

    // Routing
    if (has_role($conn, "admin")) {
        redirect('/admin/admin_dashboard.php');
    } elseif (has_role($conn, "seller")) {
        redirect('/seller/seller_dashboard.php');
    } else {
        redirect('/public/index.php');
    }
}
```

Figure 4: *OTP verification using password_verify against the stored hash; on success the OTP fields are set to NULL, the staging session key is replaced with user_id, and the session ID is regenerated.*

Role-Based Access Control

Three PHP functions in includes/auth.php enforce role separation at the page level, called at the top of every protected file before any output:

- require_login() redirects to login if no user_id is in session.
- require_role($conn, 'admin' | 'seller') calls require_login() then queries user_roles joined to roles. If the role is absent, the user is redirected to their own dashboard with a flash error.
- require_buyer_only($conn) blocks admin and seller accounts from buyer-only pages such as cart and checkout, redirecting each role to its own dashboard.

```php
function require_role(mysqli $conn, string $role_name): void {
Click to collapse the range.

  if (!has_role($conn, $role_name)) {
    flash_set('danger', 'You do not have permission to access that page.');

    $r = get_primary_role($conn);
    if ($r === 'admin') redirect('/admin/admin_dashboard.php');
    if ($r === 'seller') redirect('/seller/seller_dashboard.php');
    redirect('/public/index.php');
  }
}
```

Figure 5: *require_role() enforcing role-based page access with redirect logic, and has_role() performing a live parameterised JOIN query against user_roles and roles on every request.*

E-Commerce Features

Product Catalogue and Search

The store page (public/store.php) queries active products with stock > 0 and supports free-text search on the title column using a LIKE parameter and category filtering via a LEFT JOIN on product_categories. Both filters are appended as parameterised conditions; the types string and parameter array are built dynamically. Results are ordered by creation date descending.

```php
$search = trim($_GET['search'] ?? '');
$category_id = (int)($_GET['category_id'] ?? 0);

$params = [];
$types = "";

// Include image columns
$sql = "SELECT DISTINCT
          p.product_id, p.title, p.price, p.stock, p.product_condition, p.created_at,
          p.image_path, p.image_alt,
          u.full_name AS seller_name
        FROM products p
        JOIN users u ON u.user_id = p.seller_id
        LEFT JOIN product_categories pc ON pc.product_id = p.product_id
        WHERE p.status = 'active' AND p.stock > 0";

if ($search !== '') {
  $sql .= " AND p.title LIKE ?";
  $params[] = "%" . $search . "%";
  $types .= "s";
}

if ($category_id > 0) {
  $sql .= " AND pc.category_id = ?";
  $params[] = $category_id;
  $types .= "i";
}

$sql .= " ORDER BY p.created_at DESC";

$stmt = mysqli_prepare($conn, $sql);
if ($params) {
  mysqli_stmt_bind_param($stmt, $types, ...$params);
}
mysqli_stmt_execute($stmt);
$res = mysqli_stmt_get_result($stmt);
```

Figure 6: *Dynamic parameterised product query with optional LIKE search and category filter appended conditionally, bound using a dynamically built types string and the spread operator.*

Shopping Cart

The cart is session-backed: $_SESSION['cart'] is a PHP associative array mapping product_id to quantity. For persistence across browser restarts, the cart is synchronised to a 30-day cookie (gebeya_cart, JSON-encoded) by functions.php on every request. On page load, if the session cart is absent but the cookie is present, the cookie value is decoded and restored to the session.

```php
// Persistent Cart Logic
if (isset($_SESSION['cart'])) {
  // Sync current session cart to cookie (valid for 30 days)
  setcookie('gebeya_cart', json_encode($_SESSION['cart']), time() + 86400 * 30, '/');
} elseif (isset($_COOKIE['gebeya_cart'])) {
  // Restore cart from cookie if session cart is not set
  $decoded = json_decode($_COOKIE['gebeya_cart'], true);
  if (is_array($decoded)) {
    $_SESSION['cart'] = $decoded;
  } else {
    $_SESSION['cart'] = [];
  }
}
```

Figure 7: *Cart persistence logic syncing the session cart to a 30-day browser cookie on every request, and restoring it from the cookie when the session cart is absent.*

Checkout and Order Creation

The checkout page wraps the entire order creation process in a MySQLi transaction (mysqli_begin_transaction). Inside the transaction:

- An order row is inserted with the buyer's ID, selected address ID, and the server-calculated total amount.
- Each cart item is inserted into order_items with its product_id, seller_id, quantity, and unit and line prices as recorded at the time of purchase.
- Stock is decremented atomically with UPDATE products SET stock = stock - ? WHERE product_id = ? AND stock >= ?. If affected_rows is not exactly 1, an exception is thrown and the transaction rolls back.
- A payment row is inserted. For mobile_money and card methods, payment_status is set to 'paid' immediately and order_status is updated to 'paid' in the same transaction.

```php
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
  if (!$errors) {
    mysqli_begin_transaction($conn);

    try {
      // Create order
      $stmt = mysqli_prepare($conn,
        "INSERT INTO orders (buyer_id, address_id, order_status, total_amount)
         VALUES (?, ?, 'pending', ?)"
      );
      mysqli_stmt_bind_param($stmt, "iid", $buyer_id, $address_id, $total);
      mysqli_stmt_execute($stmt);
      $order_id = mysqli_insert_id($conn);
      mysqli_stmt_close($stmt);

      // Insert items + reduce stock
      $oi = mysqli_prepare($conn,
        "INSERT INTO order_items
         (order_id, product_id, seller_id, quantity, unit_price, line_total, item_status
         VALUES (?, ?, ?, ?, ?, ?, 'pending')"
      );

      $stock_stmt = mysqli_prepare($conn,
        "UPDATE products SET stock = stock - ?
         WHERE product_id = ? AND stock >= ?"
      );

      foreach ($items as $it) {
        mysqli_stmt_bind_param(
          $oi,
          "iiiidd",
          $order_id,
          $it['product_id'],
          $it['seller_id'],
          $it['qty'],
          $it['unit_price'],
          $it['line_total']
        );
        mysqli_stmt_execute($oi);
```

```php
$_SERVER['REQUEST_METHOD'] === 'POST') {
  (!$errors) {
try {


  // Payment simulation
  $payment_status = in_array($payment_method, ['mobile_money','card']) ? 'paid' : 'pendi
  $paid_at = ($payment_status === 'paid') ? date('Y-m-d H:i:s') : null;

  $stmt = mysqli_prepare($conn,
    "INSERT INTO payments (order_id, payment_method, amount, payment_status, paid_at)
     VALUES (?, ?, ?, ?, ?)"
  );
  mysqli_stmt_bind_param($stmt, "isdss",
    $order_id,
    $payment_method,
    $total,
    $payment_status,
    $paid_at
  );
  mysqli_stmt_execute($stmt);
  mysqli_stmt_close($stmt);

  if ($payment_status === 'paid') {
    $up = mysqli_prepare($conn, "UPDATE orders SET order_status='paid' WHERE order_id=?"
    mysqli_stmt_bind_param($up, "i", $order_id);
    mysqli_stmt_execute($up);
    mysqli_stmt_close($up);
  }

  mysqli_commit($conn);

  $_SESSION['cart'] = [];
  flash_set('ok', 'Order placed successfully!');
  redirect('/public/my_orders.php');

} catch (Throwable $e) {
  mysqli_rollback($conn);
  $errors[] = "Checkout failed. Try again.";
}
```

Figure 8: *Full checkout transaction wrapping order creation, order item insertion, atomic stock decrement with an affected_rows guard, and payment recording rolling back automatically on any failure.*

Seller Listing Management

Sellers create listings through listing_form.php which validates title, price (> 0), stock (>= 0), condition, and at least one category. Image uploads are validated first by checking the PHP file error flag, then by using finfo_file() to read the binary MIME type from the temporary file path only image/jpeg, image/png, and image/webp are accepted. The file is only moved to uploads/products/ after the database transaction commits. If the transaction rolls back, the newly uploaded file is deleted to prevent orphaned files.

```php
/* 3) Upload image (optional) AFTER product exists */
if (!empty($_FILES['image']['name'])) {
  if ($_FILES['image']['error'] !== UPLOAD_ERR_OK) {
    throw new Exception("Image upload failed.");
  }

  $tmp  = $_FILES['image']['tmp_name'];
  $size = (int)$_FILES['image']['size'];

  if ($size > 2 * 1024 * 1024) {
    throw new Exception("Image too large (max 2MB).");
  }

  $finfo = finfo_open(FILEINFO_MIME_TYPE);
  $mime  = finfo_file($finfo, $tmp);
  finfo_close($finfo);

  $allowed = [
    'image/jpeg' => 'jpg',
    'image/png'  => 'png',
    'image/webp' => 'webp',
  ];

  if (!isset($allowed[$mime])) {
    throw new Exception("Invalid image type. Use JPG, PNG, or WebP.");
  }

  $ext = $allowed[$mime];

  $destDir = __DIR__ . '/../uploads/products';
  if (!is_dir($destDir)) {
    @mkdir($destDir, 0755, true);
  }
  if (!is_dir($destDir) || !is_writable($destDir)) {
    throw new Exception("Upload folder not writable: " . $destDir);
  }
```

Figure 9: *Image upload pipeline validating the file error flag and size limit, detecting MIME type from the binary header via finfo, constructing a safe filename, and saving the file only after the database transaction commits.*

Order Fulfilment

Sellers manage their own order items through seller/orders.php. Item status transitions (pending → packed → shipped → cancelled) are applied via a CSRF-protected POST form. The UPDATE statement includes AND seller_id = ? so a seller can only change the status of items belonging to their own products.

```php
if ($_SERVER['REQUEST_METHOD'] === 'POST' && ($_POST['action'] ?? '') === 'update_status') {
  check_csrf();

  $order_item_id = (int)($_POST['order_item_id'] ?? 0);
  $new_status = $_POST['item_status'] ?? 'pending';

  $valid = ['pending','packed','shipped','cancelled'];

  if ($order_item_id > 0 && in_array($new_status, $valid, true)) {

    $stmt = mysqli_prepare(
      $conn,
      "UPDATE order_items
       SET item_status=?
       WHERE order_item_id=? AND seller_id=?"
    );

    mysqli_stmt_bind_param($stmt, "sii", $new_status, $order_item_id, $seller_id);
    mysqli_stmt_execute($stmt);
    mysqli_stmt_close($stmt);

    flash_set('ok', 'Item status updated.');
  }

  redirect('/seller/orders.php');
}
```

Figure 10: *Seller order status update handler calling check_csrf(), whitelisting the new status with in_array, and issuing an UPDATE that includes AND seller_id = ? to enforce ownership at the query level.*

Input Validation and Output Encoding

Server-Side Validation

All user-supplied values are validated before any database interaction. The pattern used across every form is collect, validate, then act. If any validation error is found, the form is re-rendered with error messages and the submitted values are preserved for correction.

- Email addresses: validated with filter_var($email, FILTER_VALIDATE_EMAIL).
- Passwords: minimum 8 characters, checked against four regex patterns (uppercase, lowercase, digit, special character) on registration and password change.
- Full names: strip_tags() and a regex removal of <script>/<style> blocks applied before storage.
- Account type: verified with in_array($account_type, ['buyer', 'seller'], true) to reject any unlisted value.
- Product condition and item status: whitelisted with in_array() before any UPDATE is issued.
- Payment method: whitelisted against the four supported values before insertion.
- Address delete: address_id cast to int; DELETE requires AND user_id = ? to prevent cross-account deletion.

```php
// Password complexity check
if (strlen($password) < 8) {
  $errors[] = "Password must be at least 8 characters.";
} elseif (!preg_match('/[A-Z]/', $password) || !preg_match('/[a-z]/', $password) || !preg_match('/[0-9]/', $password) || !preg_match('/[^a-zA-Z0-9]/', $password)) {
  $errors[] = "Password must contain at least one uppercase letter, one lowercase letter, one number, and one special character.";
}

if (!in_array($account_type, ["buyer", "seller"], true)) $errors[] = "Invalid account type.";
```

Figure 11: *Server-side password complexity enforcement using strlen and four preg_match checks, followed by a strict in_array whitelist on the account type field.*

Output Encoding

All user-controlled data rendered in HTML passes through the global e() function defined in includes/functions.php:

```php
function e(string $s): string {
  return htmlspecialchars($s, ENT_QUOTES, 'UTF-8');
}
```

Figure 12: *The global e() output encoding helper wrapping htmlspecialchars with ENT_QUOTES, shown in use on product titles, seller names, and image attributes in the store template.*

Frontend Architecture

The frontend is built without a JavaScript framework. A single shared stylesheet (assets/css/style.css) and a shared script (assets/js/main.js) are loaded on every page via the header and footer includes. All user interaction is handled through standard HTML forms and server-side redirects; no AJAX calls are made.

Pages and Layout

All pages use a two-file wrapper: includes/header.php outputs the DOCTYPE, head block, and navigation; includes/footer.php closes the container. The header conditionally renders navigation items based on the user's logged-in state and primary role, using has_role() to determine which links are shown.

```php
$isLoggedIn = !empty($_SESSION['user_id']);
$isAdmin  = $isLoggedIn ? has_role($conn, 'admin')  : false;
$isSeller = $isLoggedIn ? has_role($conn, 'seller') : false;

// In single-role mode, buyer is "not admin and not seller"
$isBuyer = $isLoggedIn && !$isAdmin && !$isSeller;
```

Figure 13: *Conditional navigation rendering using $isAdmin, $isSeller, and $isBuyer flags derived from live has_role() checks, serving a different set of links to each role and to unauthenticated guests.*

Flash Message System

flash_set('ok' | 'danger', $message) writes to $_SESSION['flash']. The header partial calls flash_get_all(), which reads and immediately unsets the array. Messages survive exactly one redirect and are never shown twice.

JavaScript

The main.js file provides a mobile navigation toggle, real-time cart counter updates, and a search bar reveal animation. No sensitive logic lives in the client; role checks, data access, and all business rules are enforced server-side.

Performance Considerations

Database indexes are placed on the columns most frequently used in WHERE clauses: email is UNIQUE on users, title and status are indexed on products, buyer_id and order_status are indexed on orders, and seller_id and order_id are indexed on order_items. The products table also indexes seller_id for fast retrieval of a seller's own listings.

All database access uses MySQLi prepared statements. Statement handles are reused inside loops; for example, inserting multiple order items or product categories in a single transaction uses one prepared statement executed multiple times, avoiding repeated query parsing overhead.

Image files are served directly from the uploads/products/ directory as static files with no PHP overhead. The loading="lazy" attribute is applied to all product thumbnails and detail images to defer off-screen image loading in the browser.

Conclusion

Gebeya is a full-stack e-commerce marketplace where the shopping experience and the security system are tightly integrated. A visitor can browse the catalogue without an account, but any purchase-related action requires completing both factors of the MFA login flow first.

The three-role architecture is enforced at the PHP layer through parameterised role-check functions that query the database on every request, with ownership checks embedded in every UPDATE and DELETE that handles user-owned data. The codebase applies a consistent pattern of validate, prepare, and execute throughout all form handlers, and every piece of user-supplied data is passed through the e() output encoder before being rendered.

The main areas identified for future development are automated testing coverage, rate limiting on the registration endpoint, a product search and advanced filter UI, a real payment gateway integration, and WebAuthn support for hardware security keys.